Hands-On Activity 6 | Data Visualization and Design

Name/s :

Canoy, Hail

Corro, Yazinel

De Leon, Benj Federic

Course Code and Title : CPE 031 - Visualizations and Data Analysis

Date Submitted : September 4, 2026

Instructor : Engr. Jimlord M. Quejado

---

1. Objectives:

This activity aims to demonstrate students' ability to apply data visualization principles, such as preattentive attributes, Gestalt principles, data-ink ratio, and common design mistakes, in creating clear and effective charts and graphs.

2. Intended Learning Outcomes (ILOs):

By the end of this module the students should be able to demonstrate:
1. Analyze data charts in various use cases.
2. Create effective data visualizations using appropriate chart types.
3. Apply principles such as preattentive attributes, Gestalt principles, and data-ink ratio to enhance chart clarity.

3. Discussions:

In this chapter, we explore key design elements that contribute to effective data visualizations. However, creating a good visualization isn't just about following a set of steps it's about addressing the needs of the audience. The first step in designing an effective chart or table is understanding its purpose and the audience's needs. Once that's clear, the design elements discussed in this chapter can be applied to best serve the visualization's purpose.

We begin by examining preattentive attributes, the visual features our brains process automatically. We then introduce Gestalt principles, which explain how people perceive and interpret visual information. These concepts are used to enhance visualizations by reducing clutter and increasing the data-ink ratio. The chapter also emphasizes minimizing the audience's eye movement and selecting appropriate fonts to ensure the visualization is easy to interpret. Lastly, we cover common mistakes in data visualization design and offer strategies to avoid them.

4. Procedures:

Preattentive Attributes

Preattentive attributes are visual features that our brain processes effortlessly and almost instantly—without conscious effort—via iconic memory. Leveraging these attributes in data visualizations helps reduce cognitive load and quickly draw attention to the most important elements.

Four Key Types of Preattentive Attributes

1. Color

Color is one of the most powerful preattentive attributes because it stands out immediately in a visual field.

Hue: Refers to the "name" of the color (e.g., red, blue, green). Hue is often used to group categories or highlight distinctions. Example: red dots among gray dots are instantly noticeable.

Saturation: Refers to the intensity of a color. A bright red circle pops out against dull pink ones, even if they share the same hue.

Luminance (Brightness): Refers to how light or dark a color appears. A light yellow square will stand out against dark gray ones.



2. Form

Form refers to how an object's visual shape and structure influence perception. Several sub-attributes fall here:

Orientation: Angles or directions can make certain elements stand out (e.g., a diagonal line among horizontal ones).

Size: Bigger objects capture attention first. Example: a large bubble in a bubble chart stands out immediately.

Shape: Different shapes can indicate grouping. Example: using triangles vs circles in a scatter plot to show two categories.

Length: Humans are very good at comparing lengths (bar chart bars), making length a highly effective way to show quantitative differences.

Width: Thicker or thinner elements can communicate weight or importance (e.g., flow widths in Sankey diagrams).



3. Spatial Positioning

Spatial positioning refers to where elements are placed in relation to each other. Humans are naturally skilled at noticing patterns, clustering, and alignment in space.

* X and Y positions: In scatter plots, our eyes instantly see clusters, trends, or outliers.
* Alignment: Objects aligned differently from others draw attention.
* Grouping: Proximity of objects suggests relationships (Gestalt principle of proximity).

Orientation

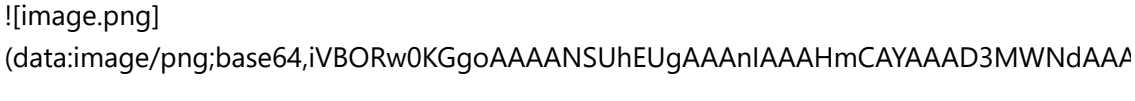


Size



Shape

-Grouping items by color and shape



4. Movement

Movement is a strong but less commonly used preattentive attribute (especially in static charts). Our brains are highly sensitive to motion—it immediately captures attention.

* Flicker/Animation: Blinking elements or pulsing dots draw the eye faster than color or size changes.
* Motion Path: Direction of movement can show flow or change over time.
* Dynamic Visualization: Used more in digital dashboards and interactive graphics rather than static charts.

Coding Procedure

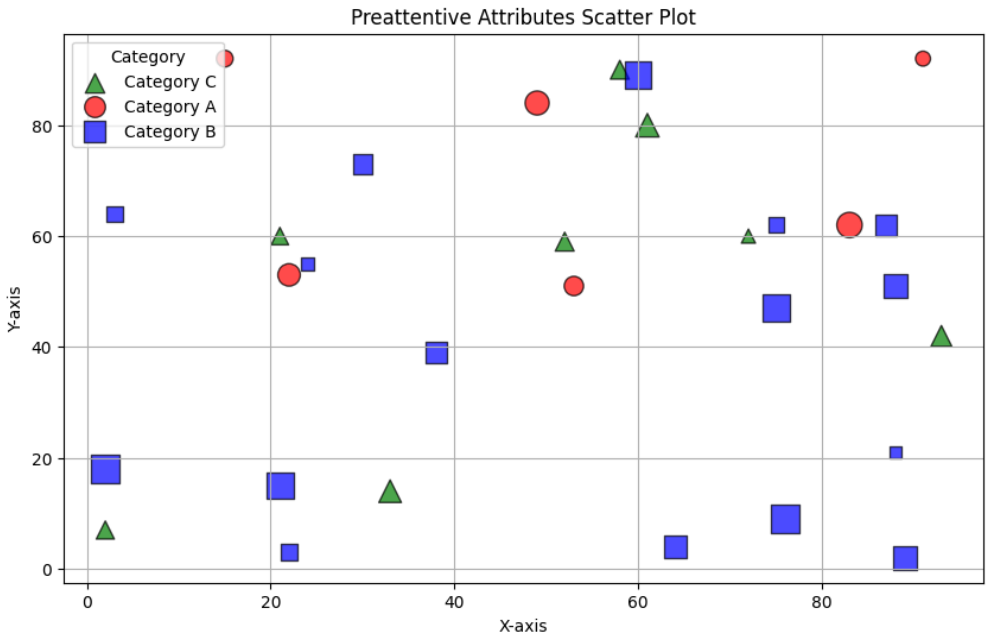
In [65]:
```
# Import libraries
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

In [66]:
```
# Step 1: Sample data
np.random.seed(42)
data = pd.DataFrame({
    'X': np.random.randint(1, 100, 30),
    'Y': np.random.randint(1, 100, 30),
    'Category': np.random.choice(['A', 'B', 'C'], 30),
    'Size': np.random.randint(50, 300, 30)
})
```

In [67]:
```
# Step 2: Map categories to colors and shapes
colors = {'A': 'red', 'B': 'blue', 'C': 'green'}
shapes = {'A': 'o', 'B': 's', 'C': '^'}
```

In [68]:
```
# Step 3: Plot scatter points for each category
plt.figure(figsize=(10,6))
for cat in data['Category'].unique():
    subset = data[data['Category'] == cat]
    plt.scatter(subset['X'], subset['Y'],
                s=subset['Size'],        # Size of points
                c=colors[cat],           # Color by category
                marker=shapes[cat],      # Shape by category
                alpha=0.7,
                edgecolors='black',
                label=f'Category {cat}')

# Step 4: Add labels, title, legend
plt.title('Preattentive Attributes Scatter Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend(title='Category')
plt.grid(True)
plt.show()
```



Task 1 : Use preattentive attributes (color, shape, and size) to plot a scatter plot.

Map:

Group → color and shape

Score → size of the markers

Add title, axis labels, and legend.

Try to make the visualization clear and easy to interpret at a glance.

In [69]:
```
# Step 1: sample dataset 2
np.random.seed(101)  # Different seed for variety
sample_data = pd.DataFrame({
    'Height': np.random.randint(150, 200, 25),       # Heights in cm
    'Weight': np.random.randint(50, 100, 25),        # Weights in kg
    'Group': np.random.choice(['X', 'Y', 'Z'], 25),  # Different categories
    'Score': np.random.randint(10, 100, 25)          # Some numerical value
})
```

In [70]:
```
# Step 2: Preview the sample data
print(sample_data)
```

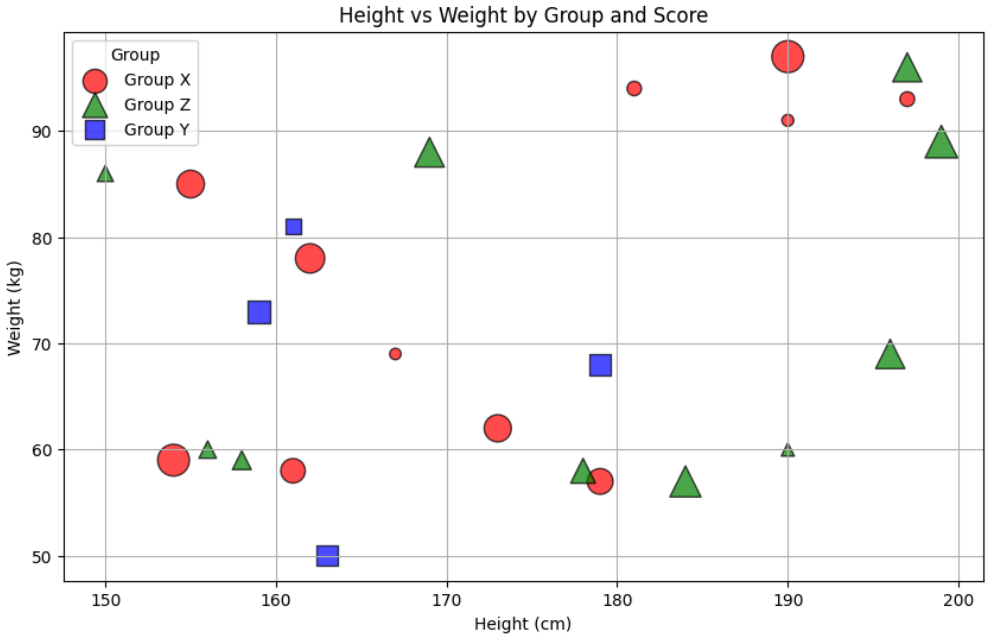
```
    Height  Weight Group  Score
0      181      94     X     19
1      161      58     X     55
2      167      69     X     12
3      156      60     Z     28
4      173      62     X     68
5      161      81     Y     21
6      197      93     X     20
7      159      73     Y     45
8      163      50     Y     38
9      190      91     X     13
10     154      59     X     93
11     190      97     X     94
12     178      58     Z     57
13     150      86     Z     24
14     196      69     Z     79
15     155      85     X     70
16     162      78     X     79
17     179      57     X     61
18     190      60     Z     16
19     199      89     Z     98
20     169      88     Z     81
21     197      96     Z     78
22     158      59     Z     33
23     179      68     Y     45
24     184      57     Z     89
```

Task 1

In [71]:
```
colors = {'X': 'red', 'Y': 'blue', 'Z': 'green'}
shapes = {'X': 'o', 'Y': 's', 'Z': '^'}
```

In [72]:
```
#Plot scatter points for each category
plt.figure(figsize=(10, 6))
for grp in sample_data['Group'].unique():
    subset = sample_data[sample_data['Group'] == grp]
    plt.scatter(subset['Height'], subset['Weight'],
                s=subset['Score'] * 4,       # Size mapped to Score (scaled
for visibility)
                c=colors[grp],                # Color by Group
                marker=shapes[grp],           # Shape by Group
                alpha=0.7,
                edgecolors='black',
                label=f'Group {grp}')

#Add title, axis labels, legend
plt.title('Height vs Weight by Group and Score')
plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.legend(title='Group')
plt.grid(True)
plt.show()
```



Gestalt Principles

refer to the guiding principles of how people interpret and perceive what they see. These principles can be used in the design of effective data visualizations. The principles generally describe how people define order and meaning in things that they see. We will limit our discussion to the four Gestalt principles that are most closely related to the design of data visualizations: similarity, proximity, enclosure, and connection. An understanding of these principles can help in creating more effective data visualizations and help differentiate between clutter and meaningful design in data visualizations.

1. Similarity

The Gestalt principle of similarity suggests that people naturally see objects with shared features such as color, shape, size, orientation, or other preattentive attributes as part of the same group. In data visualization, this means that when elements share these characteristics, the audience will interpret them as being related. In the example below, objects with the same color or shape are perceived as grouped together. Therefore, when designing visualizations, it is essential to use similar attributes only for elements that are meant to be grouped, to avoid misleading the audience.



2. Proximity

The Gestalt principle of proximity states that people naturally perceive objects that are close together as belonging to the same group, while objects farther apart are seen as separate groups. This principle is often evident in data visualizations such as scatter charts. For example, if a company wants to analyze its customers by age and annual income, a scatter chart of these variables can reveal natural groupings. In the example below, we tend to see three distinct clusters of customers based on how close the points are to each other. This illustrates the Gestalt principle of proximity.



3. Enclosure

The Gestalt principle of enclosure states that objects that are physically enclosed together are perceived as belonging to the same group. This can be illustrated in two ways. First, we can reinforce the similarity principle by enclosing points that are already close to one another, as shown below. Alternatively, if there is a third attribute of the customers such as educational background that we want to highlight, we can use enclosure to indicate this grouping even when the customers are not close together in the chart. Enclosure can be represented in different ways, such as shaded areas or dashed boxes. In most cases, even a simple suggestion of enclosure is enough for the audience to perceive the objects as part of the same group.

4. Connection

The Gestalt principle of connection suggests that people perceive objects that are linked or connected as part of the same group. A common application of this principle in data visualization is in time-series data. For example, a data center company may want to compare its forecasted server loads with the actual loads experienced by customers over a two-week period. By connecting the data points, it becomes easier to see trends and differences between the forecasted and actual peak server loads.



In [73]:
```
# Step 1: Create sample dataset
np.random.seed(2025)
data = pd.DataFrame({
    'Age': np.random.randint(18, 60, 30),
    'Income': np.random.randint(30000, 120000, 30),
    'Education': np.random.choice(['High School', 'Bachelor', 'Master'],
30),
    'CustomerType': np.random.choice(['Regular', 'Premium', 'VIP'], 30),
    'VisitFrequency': np.random.randint(1, 15, 30)
})
```

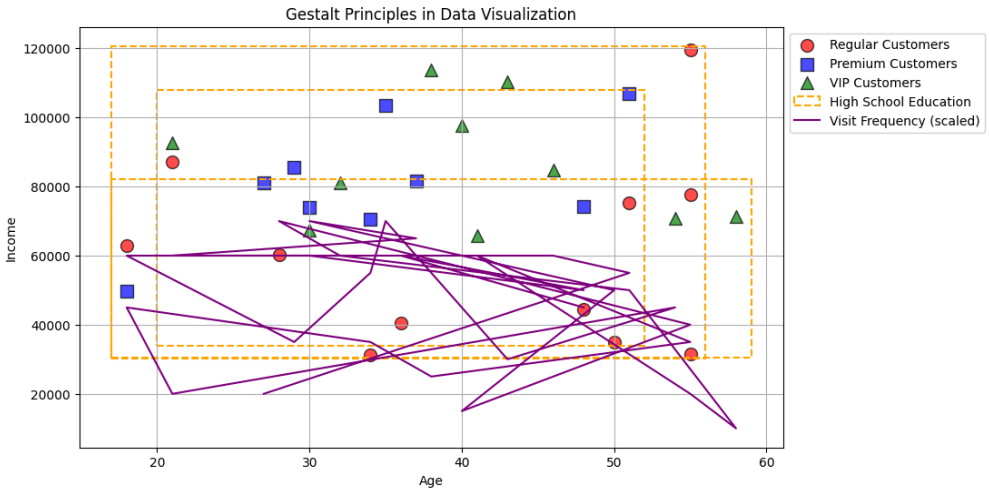
In [74]:
```
# Step 2: Map customer type to colors and shapes (Similarity)
colors = {'Regular': 'red', 'Premium': 'blue', 'VIP': 'green'}
shapes = {'Regular': 'o', 'Premium': 's', 'VIP': '^'}
```

In [75]:
```
# Step 3: Plot Age vs Income (Proximity & Similarity)
plt.figure(figsize=(10,6))
for ctype in data['CustomerType'].unique():
    subset = data[data['CustomerType'] == ctype]
    plt.scatter(subset['Age'], subset['Income'],
                c=colors[ctype],
                marker=shapes[ctype],
                s=100,
                alpha=0.7,
                edgecolors='black',
                label=f'{ctype} Customers')

# Step 4: Add Enclosure for Education groups
for edu in data['Education'].unique():
    subset = data[data['Education'] == edu]
    plt.gca().add_patch(plt.Rectangle(
        (subset['Age'].min()-1, subset['Income'].min()-1000),
        subset['Age'].max()-subset['Age'].min()+2,
        subset['Income'].max()-subset['Income'].min()+2000,
        fill=False, edgecolor='orange', linestyle='--', linewidth=1.5,
        label=f'{edu} Education' if edu=='High School' else ""
    ))

# Step 5: Connection - Visit frequency over customers
plt.plot(data['Age'], data['VisitFrequency']*5000, color='purple',
linestyle='-', linewidth=1.5, label='Visit Frequency (scaled)')

# Step 6: Add labels, title, legend
plt.title('Gestalt Principles in Data Visualization')
plt.xlabel('Age')
plt.ylabel('Income')
plt.legend(loc='upper left', bbox_to_anchor=(1,1))
plt.grid(True)
plt.show()
```



Task 2 :

Similarity: Use color or shape to distinguish WorkoutType.

Proximity: Plot Steps vs CaloriesBurned and identify natural clusters.

Enclosure: Highlight groups based on Mood using enclosed shapes (like circles, rectangles, or shaded areas).

Connection: Create a line plot showing Steps over Day.

In [76]:
```
# Step 1: Create sample dataset
np.random.seed(404)
fitness_data = pd.DataFrame({
    'Day': np.arange(1, 31),                                # Days of the
month
    'Steps': np.random.randint(3000, 15000, 30),          # Daily steps
    'CaloriesBurned': np.random.randint(200, 800, 30),    # Daily calories
burned
    'WorkoutType': np.random.choice(['Cardio', 'Strength', 'Yoga'], 30),
    'Mood': np.random.choice(['Happy', 'Neutral', 'Tired'], 30)  # Mood
after workout
})
```

In [77]:
```
# Step 2: Preview the data
print(fitness_data)
```

	Day	Steps	CaloriesBurned	WorkoutType	Mood
0	1	10911	281	Strength	Neutral
1	2	8884	426	Strength	Tired
2	3	8302	263	Strength	Neutral
3	4	8108	299	Strength	Tired
4	5	9031	518	Strength	Tired
5	6	7721	600	Cardio	Neutral
6	7	5657	276	Strength	Happy
7	8	5119	746	Yoga	Neutral
8	9	14771	485	Strength	Happy
9	10	6778	285	Yoga	Tired
10	11	10359	502	Strength	Tired
11	12	14111	691	Yoga	Neutral
12	13	6595	514	Strength	Happy
13	14	14737	202	Strength	Happy
14	15	8205	766	Yoga	Neutral
15	16	11248	762	Strength	Tired
16	17	13615	582	Cardio	Happy
17	18	10470	262	Strength	Happy
18	19	14031	703	Yoga	Happy
19	20	6483	589	Cardio	Neutral
20	21	7603	699	Cardio	Tired
21	22	10555	471	Yoga	Happy
22	23	8509	522	Cardio	Neutral
23	24	11723	338	Yoga	Happy
24	25	7854	359	Strength	Happy
25	26	9882	360	Strength	Happy
26	27	5830	728	Yoga	Happy
27	28	7911	412	Strength	Neutral
28	29	12983	641	Cardio	Neutral
29	30	9796	399	Cardio	Happy

Task 2

In [78]:
```
#Step 2 Map WorkoutType to colors and shapes (Similarity)
colors = {'Cardio': 'red', 'Strength': 'blue', 'Yoga': 'green'}
shapes = {'Cardio': 'o', 'Strength': 's', 'Yoga': '^'}
```

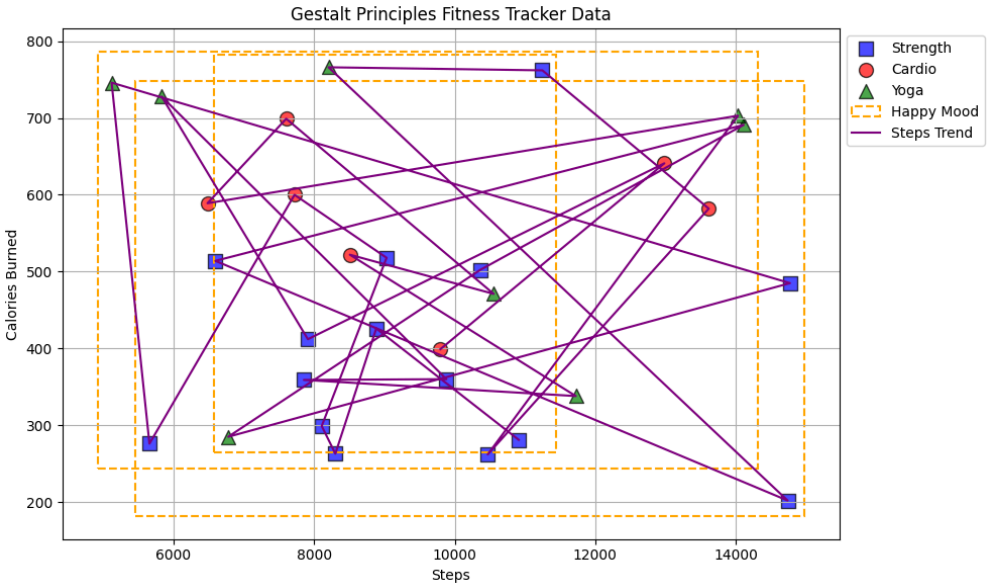
In [79]:

```
#Step 3: Plot Steps vs CaloriesBurned (Proximity & Similarity)
plt.figure(figsize=(10, 6))
for wtype in fitness_data['WorkoutType'].unique():
  subset = fitness_data[fitness_data['WorkoutType'] == wtype]
  plt.scatter(
  subset['Steps'],
  subset['CaloriesBurned'],
  c=colors[wtype],
  marker=shapes[wtype],
  s=100,
  alpha=0.7,
  edgecolors='black',
  label=f'{wtype}'
)

#Step 4: Add Enclosure for Mood groups
for mood in fitness_data['Mood'].unique():
  subset = fitness_data[fitness_data['Mood'] == mood]
  plt.gca().add_patch(plt.Rectangle(
  (subset['Steps'].min() - 200, subset['CaloriesBurned'].min() - 20),
  subset['Steps'].max() - subset['Steps'].min() + 400,
  subset['CaloriesBurned'].max() - subset['CaloriesBurned'].min() + 40,
  fill=False, edgecolor='orange', linestyle='--', linewidth=1.5,
  label=f'{mood} Mood' if mood == 'Happy' else ""
))

#Step 5: Connection - Steps over Day
plt.plot(
fitness_data['Steps'],
fitness_data['CaloriesBurned'],
color='purple',
linestyle='-',
linewidth=1.5,
label='Steps Trend'
)

#Step 6: Add labels, title, legend
plt.title('Gestalt Principles Fitness Tracker Data')
plt.xlabel('Steps')
plt.ylabel('Calories Burned')
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.grid(True)
plt.tight_layout()
plt.show()
```



Data-Ink Ratio

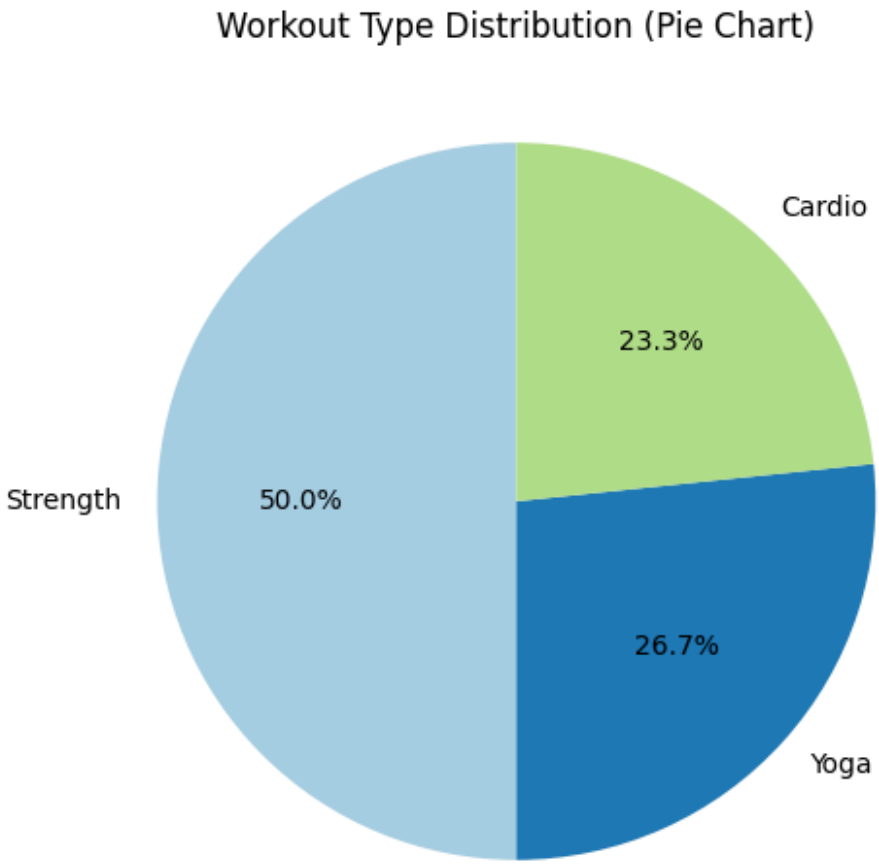
Preattentive attributes and Gestalt principles are important for understanding which features can be used to visualize data and how the brain processes these visual cues. However, it's easy to overuse these elements, which can reduce their effectiveness in highlighting and distinguishing key information. A key guideline for creating effective data visualizations is that charts and graphs should clarify the data, helping the audience gain insights and understanding. They should avoid excessive clutter that could obscure the data or make it hard to interpret.

A common way to approach this principle is through the concept of maximizing the data-ink ratio. This ratio compares the amount of "data-ink" used to the total ink in a table or chart, where data-ink refers to the ink necessary for conveying the data's meaning to the audience. Non-data-ink, on the other hand, includes elements that do not contribute to understanding the data. For example, a pie chart might use color and a legend to differentiate between categories, whereas a bar chart can convey the same information without these extra elements, resulting in a higher data-ink ratio.

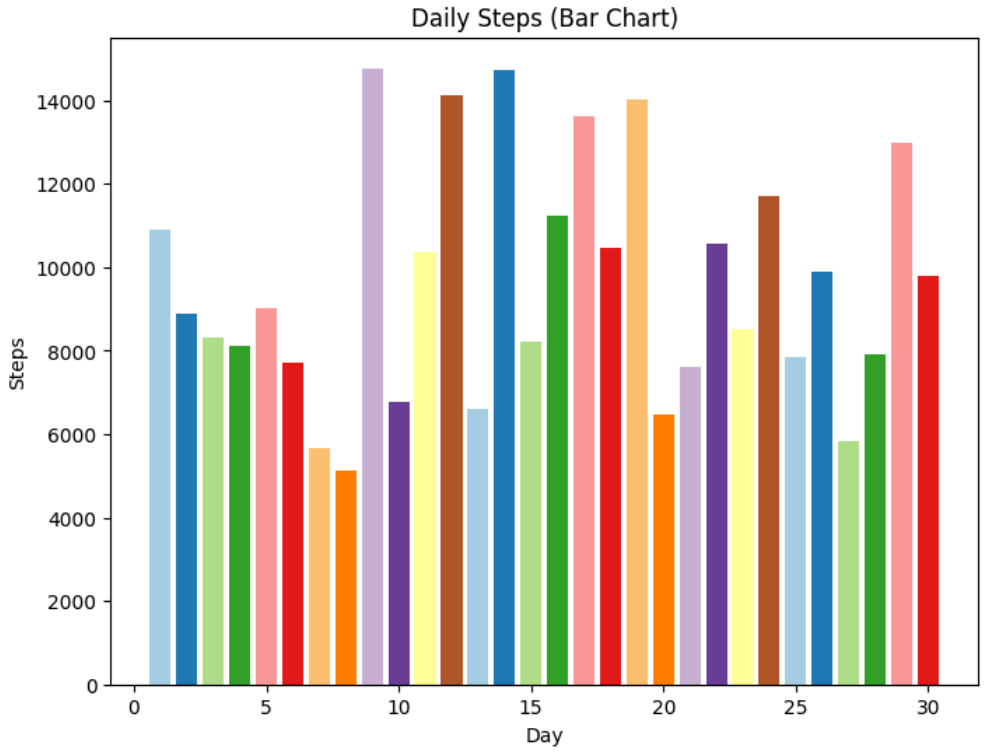
In [80]:
```
# Step 1: Create sample dataset
np.random.seed(404)
fitness_data = pd.DataFrame({
    'Day': np.arange(1, 31),                                # Days of the
month
    'Steps': np.random.randint(3000, 15000, 30),            # Daily steps
    'CaloriesBurned': np.random.randint(200, 800, 30),    # Daily calories
burned
    'WorkoutType': np.random.choice(['Cardio', 'Strength', 'Yoga'], 30),
    'Mood': np.random.choice(['Happy', 'Neutral', 'Tired'], 30)  # Mood
after workout
})
```

In [81]:
```
# Step 2: Visualize the data using a Pie Chart
workout_type_counts = fitness_data['WorkoutType'].value_counts()

plt.figure(figsize=(8, 6))
plt.pie(workout_type_counts, labels=workout_type_counts.index,
autopct='%1.1f%%', startangle=90, colors=plt.cm.Paired.colors)
plt.title('Workout Type Distribution (Pie Chart)')
plt.show()
```



In [82]:
```
# Step 3: Visualize the data using a Bar Chart
plt.figure(figsize=(8, 6))
plt.bar(fitness_data['Day'], fitness_data['Steps'],
color=plt.cm.Paired.colors)
plt.title('Daily Steps (Bar Chart)')
plt.xlabel('Day')
plt.ylabel('Steps')
plt.show()
```



In [83]:
```
# Step 4: Estimate Data-Ink Ratios (Approximate values)
# For simplicity, we assume the following:
# - Pie chart uses 3 non-data elements (legend, colors, labels)
# - Bar chart uses 2 non-data elements (axes, labels)

pie_chart_data_ink = len(workout_type_counts) / (len(workout_type_counts) +
3)  # Pie chart: 3 non-data ink elements
bar_chart_data_ink = len(fitness_data) / (len(fitness_data) + 2)  # Bar
chart: 2 non-data ink elements

# Step 6: Print Data-Ink Ratios for Pie and Bar Charts
print(f"Pie Chart Data-Ink Ratio: {pie_chart_data_ink:.2f}")
print(f"Bar Chart Data-Ink Ratio: {bar_chart_data_ink:.2f}")
```

```
Pie Chart Data-Ink Ratio: 0.50
Bar Chart Data-Ink Ratio: 0.94
```

Task 3 :

Task: Maximizing Data-Ink Ratio in Visualization

Similarity: Use color or shape to distinguish between WorkoutType categories (Cardio, Strength, Yoga).

Proximity: Create a scatter plot of Steps vs. CaloriesBurned and identify any natural clusters.

Enclosure: Highlight groups based on Mood using enclosed shapes (like circles or rectangles).

Connection: Create a line plot showing Steps over Day to visualize trends over time.

In [84]:
```
# Step 1: Create sample dataset
np.random.seed(404)
fitness_data = pd.DataFrame({
    'Day': np.arange(1, 31),                                # Days of the
month
    'Steps': np.random.randint(3000, 15000, 30),          # Daily steps
    'CaloriesBurned': np.random.randint(200, 800, 30),    # Daily calories
burned
    'WorkoutType': np.random.choice(['Cardio', 'Strength', 'Yoga'], 30),
    'Mood': np.random.choice(['Happy', 'Neutral', 'Tired'], 30)  # Mood
after workout
})
```

In [85]:
```
# Step 2: Preview the data
print(fitness_data)
```

```
    Day  Steps  CaloriesBurned WorkoutType     Mood
0     1  10911             281    Strength  Neutral
1     2   8884             426    Strength    Tired
2     3   8302             263    Strength  Neutral
3     4   8108             299    Strength    Tired
4     5   9031             518    Strength    Tired
5     6   7721             600      Cardio  Neutral
6     7   5657             276    Strength    Happy
7     8   5119             746        Yoga  Neutral
8     9  14771             485    Strength    Happy
9    10   6778             285        Yoga    Tired
10   11  10359             502    Strength    Tired
11   12  14111             691        Yoga  Neutral
12   13   6595             514    Strength    Happy
13   14  14737             202    Strength    Happy
14   15   8205             766        Yoga  Neutral
15   16  11248             762    Strength    Tired
16   17  13615             582      Cardio    Happy
17   18  10470             262    Strength    Happy
18   19  14031             703        Yoga    Happy
19   20   6483             589      Cardio  Neutral
20   21   7603             699      Cardio    Tired
21   22  10555             471        Yoga    Happy
22   23   8509             522      Cardio  Neutral
23   24  11723             338        Yoga    Happy
24   25   7854             359    Strength    Happy
25   26   9882             360    Strength    Happy
26   27   5830             728        Yoga    Happy
27   28   7911             412    Strength  Neutral
28   29  12983             641      Cardio  Neutral
29   30   9796             399      Cardio    Happy
```

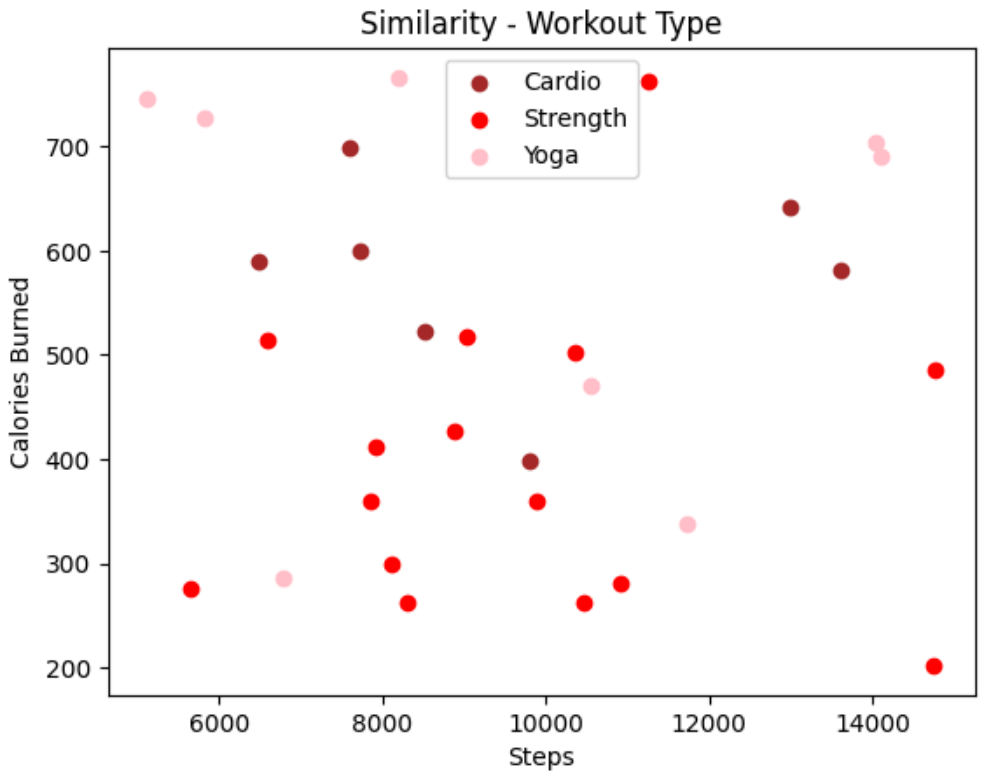
Similarity: Use color or shape to distinguish between WorkoutType categories (Cardio, Strength, Yoga).

In [86]:
```
plt.scatter(fitness_data[fitness_data['WorkoutType'] == 'Cardio']['Steps'],
fitness_data[fitness_data['WorkoutType'] == 'Cardio']['CaloriesBurned'],
color='brown',
label='Cardio')

plt.scatter(fitness_data[fitness_data['WorkoutType'] == 'Strength']
['Steps'],
fitness_data[fitness_data['WorkoutType'] == 'Strength']['CaloriesBurned'],
color='red',
label='Strength')

plt.scatter(fitness_data[fitness_data['WorkoutType'] == 'Yoga']['Steps'],
fitness_data[fitness_data['WorkoutType'] == 'Yoga']['CaloriesBurned'],
color='pink',
label='Yoga')

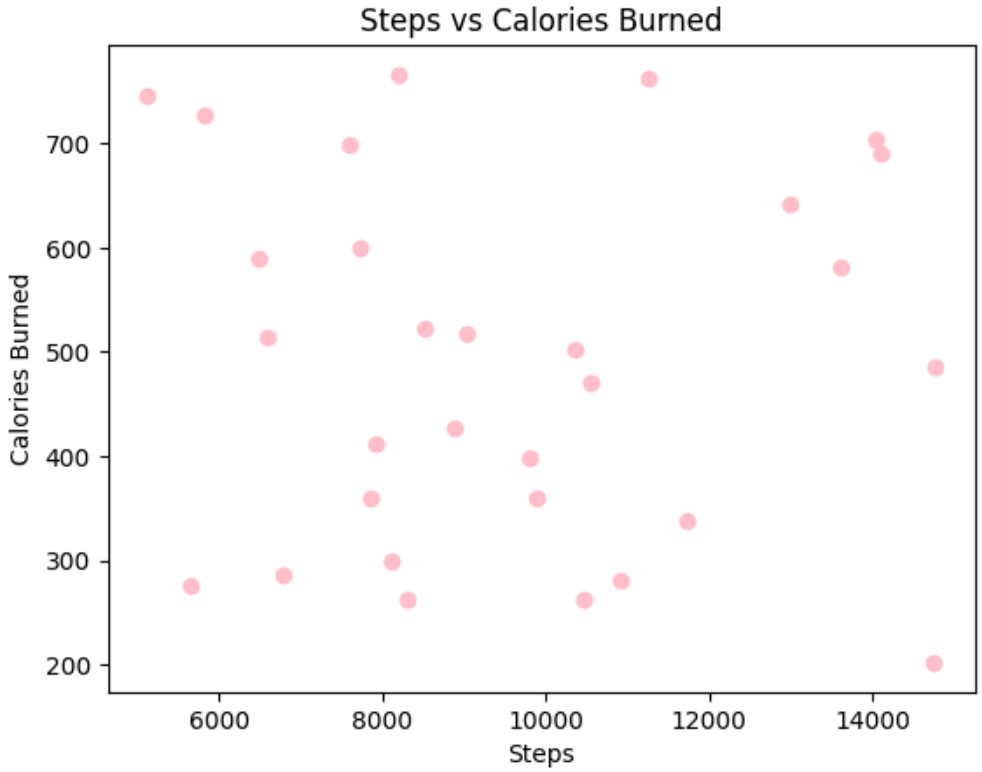
plt.xlabel('Steps')
plt.ylabel('Calories Burned')
plt.title('Similarity - Workout Type')
plt.legend()
plt.show()
```



Proximity: Create a scatter plot of Steps vs. CaloriesBurned and identify any natural clusters.

In [87]:
```
plt.scatter(fitness_data['Steps'],
            fitness_data['CaloriesBurned'],
            color='pink')


plt.xlabel('Steps')
plt.ylabel('Calories Burned')
plt.title('Steps vs Calories Burned')
plt.show()
```



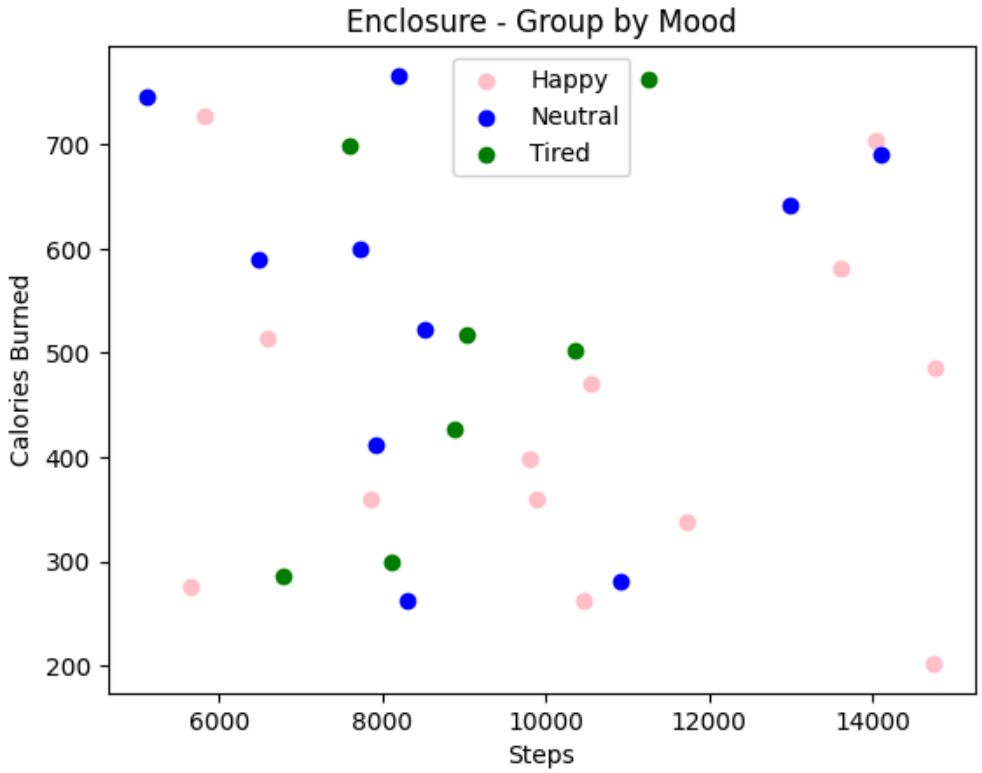
Enclosure: Highlight groups based on Mood using enclosed shapes (like circles or rectangles).

In [88]:
```
plt.scatter(fitness_data[fitness_data['Mood'] == 'Happy']['Steps'],
            fitness_data[fitness_data['Mood'] == 'Happy']
['CaloriesBurned'],
            color='pink',
            label='Happy')

plt.scatter(fitness_data[fitness_data['Mood'] == 'Neutral']['Steps'],
            fitness_data[fitness_data['Mood'] == 'Neutral']
['CaloriesBurned'],
            color='blue',
            label='Neutral')

plt.scatter(fitness_data[fitness_data['Mood'] == 'Tired']['Steps'],
            fitness_data[fitness_data['Mood'] == 'Tired']
['CaloriesBurned'],
            color='green',
            label='Tired')

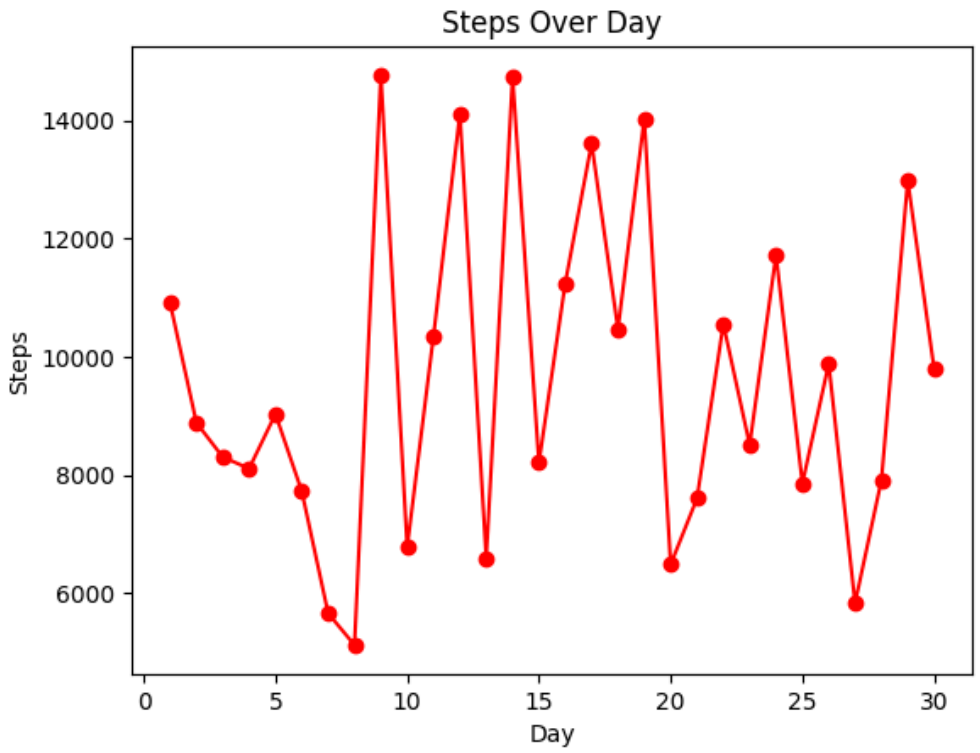
plt.xlabel('Steps')
plt.ylabel('Calories Burned')
plt.title('Enclosure - Group by Mood')
plt.legend()
plt.show()
```



Connection: Create a line plot showing Steps over Day to visualize trends over time.

In [89]:
```
plt.plot(fitness_data['Day'],
         fitness_data['Steps'],
         color='red',
         marker='o')


plt.xlabel('Day')
plt.ylabel('Steps')
plt.title('Steps Over Day')
plt.show()
```



Other Data Visualization Design Issues

Minimizing Eye Travel

ata visualizations should be easy to interpret, revealing insights while minimizing cognitive load. This can be achieved by applying preattentive attributes, Gestalt principles, and increasing the data-ink ratio. Reducing eye travel also helps in making charts more efficient.

For example, the Office of Budget and Performance Improvement for Springfield compares the clearance rates of two police districts. Below is a chart comparing the clearance rates for property crimes in Districts 1 and 2 over the last six months.

The chart features that increase eye travel, common in Excel defaults, include the legend placed at the bottom, requiring viewers to move their eyes between the chart and legend. Moving the legend closer to the lines or labeling them directly would reduce this effort. Additionally, rotated axis titles force unnecessary eye movement. Aligning titles within the chart would improve readability by minimizing eye travel.



Choosing a Font for Text

Text is a key element in data visualizations, used for labeling axes, filling table values, and highlighting important details. Since text plays such a crucial role, the choice of font is an important design consideration. Most data visualization tools, including Excel, offer a wide variety of font options.

While experts may differ on the ideal font to use, there is general consensus that some fonts are better suited for data visualizations than others. Specifically, sans-serif fonts (which lack the small projections at the end of characters) are typically preferred over serif fonts (which include these projections, called serifs). For example, common serif fonts include Times New Roman and Courier, while popular sans-serif fonts include Arial, Calibri, and Verdana.

Serif fonts are generally more suited for print, while sans-serif fonts are favored for digital displays, as they tend to be more legible at smaller sizes. Since data visualizations are often viewed both in print and digitally and involve varying font sizes, sans-serif fonts are usually the better choice. For instance, the charts in this textbook use Calibri, the default font in Excel, while printed charts use Myriad Pro due to its readability at different sizes. Other sans-serif fonts like Arial and Verdana are also acceptable.

More important than whether the font is serif or sans-serif is the consistency and restraint in font usage. Data visualizations should use the same font type and size for similar information. For example, the titles for horizontal and vertical axes should typically be in the same font and size. Inconsistent font sizes can mislead the viewer by suggesting that certain elements have more importance. Using too many different fonts can clutter the visualization and increase cognitive load. Therefore, it's best to stick to one font type, using size variations, bold, italics, or color to differentiate elements and guide the audience’s attention.



Common Mistakes in Data Visualization Design

Wrong Type of Visualization

The choice of chart or table for data visualization largely depends on the audience and the insights or message you want to convey. While this textbook offers best practices for designing effective visualizations, many decisions about which chart or design elements to use will vary depending on the specific context and goal of the visualization. In this section, we explore different situations where one type of visualization might be more appropriate than another. However, it's important to remember that the most effective visualization always depends on the audience's needs and the message being communicated.

When the goal is to present precise numerical values, tables are often a better choice than charts. Bar or column charts are typically preferred over pie charts because they make it easier for the audience to compare values, leveraging the preattentive attribute of length, which is more effective than using shape. Nonetheless, the right choice of visualization always depends on the objective, and it's not always immediately clear which one is best.

Trying to Display Too Much Information

A common mistake in creating effective charts is attempting to convey too much information on a single chart, which can overwhelm the audience by presenting too many insights at once.

For example, consider Keeland Industries, an online company that sells both OEM (original-equipment manufacturer) and aftermarket replacement parts for automobiles. Since Keeland sells parts nationwide, it divides the U.S. into 12 regions for tracking sales performance. The management team is primarily interested in comparing OEM sales across these regions to identify top performers and doing the same for aftermarket sales. Trying to combine both comparisons into one chart could lead to confusion, so it's more effective to separate these insights into distinct visualizations.

![image.png]
()

Too Many Attributes

The "Too Many Attributes" mistake occurs when a chart is overloaded with excessive visual elements, such as multiple colors, shapes, sizes, or labels, making it cluttered and difficult to interpret. This can confuse the audience and increase cognitive load, as viewers struggle to discern the most important insights. To avoid this, focus on a few key attributes and simplify the design, possibly using multiple charts to convey different insights instead of overcrowding one chart with too much information. This ensures the visualization remains clear and easy to understand.

Unnecessary use of 3D

Three-dimensional (3D) charts are often difficult for an audience to understand. Many novice analysts will create 3D charts in Excel even when the third dimension does not provide any useful information. Consider the 3D column chart shown in Figure below. This chart compares the billable hours for eight consultants who work for Stanley Consulting Group.



5. Supplementary Activity:

Instructions:

1. Select a Dataset

-Choose a .csv dataset from sources like Kaggle or Google Dataset Search.

-Define a clear problem statement that you want to analyze or visualize (e.g., relationship between Square Footage and Price).

2. Apply Preattentive Attributes

-Create a scatter plot or bar chart using color (for categories) and size/length (for numerical values). Example: Use color for Location and size for Bedrooms.

3. Apply Gestalt Principles

-Use Similarity (same color for categories), Proximity (group related data), and Enclosure (highlight specific groups). Example: Highlight high-priced houses with a shaded area.

4. Maximize Data-Ink Ratio

-Create a clean, minimalistic chart. Remove unnecessary elements like gridlines or 3D effects to focus on the data.

5. Avoid Visualization Mistakes

-Identify a mistake (e.g., wrong chart type) and correct it. For example, fix a line chart for categorical data by switching to a bar chart.

6. Provide Explanations

-Justify your choice of visualization, explain how it improves understanding, and describe any design choices (like font, color, and chart types).

Dataset: Student Performance Dataset

Problem Statement:

Many students struggle to balance their study time and sleep duration during exam preparation. While studying more generally improves grades, it is unclear how different combinations of study time and rest impact test scores.

In [90]:
```
import pandas as pd

df = pd.read_csv('/content/student_performance_dataset.csv')

df.head()
```

Out[90]:

	student_id	gender	study_time_hours	attendance_percent	sleep_hou
0	1	Male	4.0	98.0	6.5
1	2	Female	6.3	100.0	5.7
2	3	Male	4.9	85.3	7.9
3	4	Male	2.6	77.5	8.0
4	5	Male	2.2	89.6	4.6

In [91]:

```
import numpy as np
import matplotlib.pyplot as plt

#Data
study_time = ['0–3 hrs', '3–6 hrs', '6–9 hrs']
sleep_hours = ['<4 hrs', '4–6 hrs', '6–8 hrs', '8–10 hrs']

avg_scores = {
    '<4 hrs': [83, 75.5, 100],
    '4–6 hrs': [76, 85, 95],
    '6–8 hrs': [77, 86, 95],
    '8–10 hrs': [80, 89, 98]
}

#Colors are used consistently for categories (Similarity)
colors = ['red', 'blue', 'green', 'purple']

plt.figure(figsize=(10, 6))

#Proximity: bars for each study-time group are placed close together
x = np.arange(len(study_time))
width = 0.20

for i, sleep in enumerate(sleep_hours):
    plt.bar(
        x + (i - 1.5) * width,
        avg_scores[sleep],
        width,
        color=colors[i],
        label=sleep
    )

#Enclosure: highlight the 6–9 hrs study-time group
ax = plt.gca()
ax.add_patch(
    plt.Rectangle(
        (x[2] - 0.43, 0),
        0.86,
        103,
        fill=False,
        edgecolor='black',
        linestyle='--',
        linewidth=1.5
    )
)

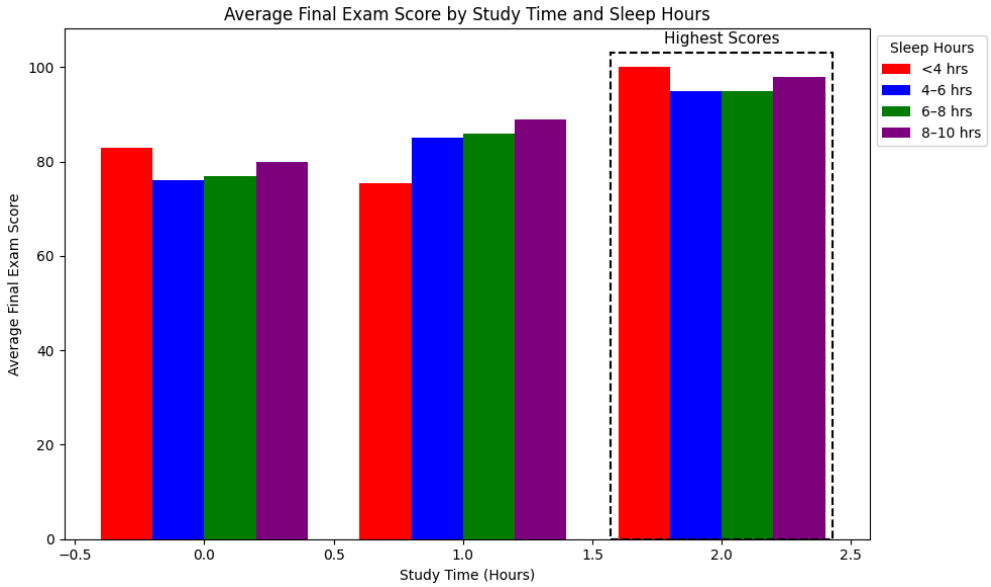
plt.text(
    x[2],
    105,
    'Highest Scores',
    ha='center',
    fontsize=11
)
```

```
#Clear labels and title
plt.title('Average Final Exam Score by Study Time and Sleep Hours')
plt.xlabel('Study Time (Hours)')
plt.ylabel('Average Final Exam Score')

#Minimal design: no gridlines or 3D effects
plt.grid(False)
plt.legend(title='Sleep Hours', loc='upper left', bbox_to_anchor=(1, 1))

plt.tight_layout()

plt.savefig('study_time_sleep_exam_score_activity6_style.png', dpi=200,
bbox_inches='tight')
plt.show()
```



Code Analysis:

The code uses NumPy and Matplotlib to create a grouped bar chart. np.arange() is used to position the study-time groups, while plt.bar() creates the bars for each sleep-hour category. Different colors are used to easily identify the sleep hours, and the bars are grouped closely to show related data. The plt.Rectangle() code adds a dashed box around the 6–9 hours group to highlight the highest scores. Finally, plt.title(), plt.xlabel(), plt.ylabel(), plt.legend(), and plt.show() are used to make the graph clear and easy to understand.

6. Conclusion/Learnings/Analysis:

The bar chart was chosen because it makes it easy to compare the average final exam scores for different study times and sleep hours. The use of different colors helps separate the sleep-hour categories, while the bar length makes the higher and lower scores easy to see. The bars are grouped closely to show proximity, and the dashed box uses enclosure to highlight the 6–9 hours study group. I also used a simple font, clear labels, and no unnecessary gridlines or 3D effects to keep the chart clean and easy to understand.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.